methods in place of Razor Pages and page handlers. Razor Pages builds directly on top of the underlying ASP.NET Core MVC framework, using the MVC framework under the hood for their behavior.

If you prefer, you can avoid Razor Pages and work with the MVC framework directly in ASP.NET Core. This option was the only one in early versions of ASP.NET Core and the previous version of ASP.NET.

TIP I look in greater depth at choosing between Razor Pages and the MVC framework in chapter 19.

In this section we look in greater depth at how the MVC design pattern applies to Razor Pages in ASP.NET Core. This section will also help clarify the role of various features of Razor Pages.

Do Razor Pages use MVC or MVVM?

Occasionally, I've seen people describe Razor Pages as using the Model-View-View Model (MVVM) design pattern rather than the MVC design pattern. I don't agree, but it's worth being aware of the differences.

MVVM is a UI pattern that is often used in mobile apps, desktop apps, and some client-side frameworks. It differs from MVC in that there is a bidirectional interaction between the view and the view model. The view model tells the view what to display, but the view can also trigger changes directly on the view model. It's often used with two-way data binding where a view model is bound to a view.

Some people consider the Razor Pages `PageModel` to be filling this role, but I'm not convinced. Razor Pages definitely seems based on the MVC pattern to me (it's based on the ASP.NET Core MVC framework after all!), and it doesn't have the same two-way binding that I would expect with MVVM.